2400-DS1AL # Algorithms for Data Science

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 3: Analyzing Algorithms

In this lecture, we examine an interesting computational problem. This rather not-well known problem is a nice example of what we could get into while analyzing data. Step by step, we will speed up our initial algorithm, each time verifying the improvements by analyzing its running time and memory complexity. Along the way, we learn two useful (and cool) techniques: *counting* and *cumulative sums*.

Recap: Analyzing Running Time

In the last lecture, we defined the running time of an algorithm as the *asymptotic worst-case number of elementary (RAM) operations*. To write the running time, we use one of the three asymptotics symbols O, Ω and Θ. In a hand-waving informal way, the running time of an algorithm is

- $O(f(n))$ if $f(n)$ is an **upper bound** on the actual running time,
- $\Omega(f(n))$ if $f(n)$ is a **lower bound** on the actual running time,
- $\Theta(f(n))$ if $f(n)$ is (asymptotically) **equal** to the actual running time.

Please see the last lecture for a precise and formal definition.

In practice, in most cases only the O-notation is used; sometimes (by mistake) even in place of the Θ-notation. The reason is that it is easier to prove just a rough upper bound on the running time than the precise asymptotic Θ-running time. However, in this lecture, we will be precise and try to use Θ. (As an exception, we might for simplicity write $O(1)$ instead of Θ(1) as the former running time automatically implies the latter one since any algorithm takes at least one step, hence, has time Ω(1).)

Multiple Parameters

Until now, we defined the running time in dependence of a single n, the "size" of the input instance. The size can be defined, for example, as the total number of input values. Sometimes, however, to get a more detailed comparison between two algorithms, we consider the running time in dependence of several parameters. For instance, if the input consists of two arrays of different lengths k and m, we could either analyze the running time with respect to $n=k+m$ (total number of values), or with respect to k and m simultaneously as a function $f(k,m)$.

For example, consider the following two (nonsense) algorithms in Python.

```
tab = [i for i in range(m*m + k)]
```

```
tab = [i for i in range(m + k*k)]
```

Their running times are asymptotic to the number of for-loop repetitions. For the first algorithm, we get $\Theta(m^2+k)$, for the second, we get $\Theta(m+k^2)$. Thus, if we knew for example that in our application m is always much larger than k, we would prefer the second algorithm to the first one. By analyzing the running time with respect to only $n=k+m$, we get for both algorithms the same running time of $\Theta(n^2)$ (as the worst-case for the first algorithm is when $k=0$ and thus $n=m$, for the second one when $m=0$ and thus $n=k$) making the two algorithms indistinguishably to us.

A formal definition of the running time depending on several parameters is discussed in the exercises.

Example Problem: Counting Points in Many Overlapping Rectangles

Let us now turn to the main problem of this lecture: geometrically, we are given n points and k rectangles, and our task is to count the number of points in each rectangle. We can have multiple points with the same coordinates.

INPUT:

- Integer arrays x and y of size n.
- Integer arrays x0, y0, x1, y1 of size k.
- We know that

- all values in x,x0,x1 are between 0 and W-1,
- all values in y,y0,y1 between 0 and H-1,
- for all j, x0[j]≤x1[j] and y0[j]≤y1[j].

OUTPUT:
An array res of size k such that res[i] is the number of j such that x0[i]≤x[j]≤x1[i] and y0[i]≤y[j]≤y1[i].

Geometrically, the arrays x and y represent n points: the i-th point has the coordinates (x[i],y[i]).
The arrays x0, y0, x1, y1 represent k rectangles: the four corner points of the i-th rectangle are (x0[i],y0[i]), (x1[i],y0[i]), (x0[i],y1[i]), (x1[i],y1[i]).
The values W and H bound the size of the whole screen containing all the points and rectangles.

Our "intended" application: you are writing an eye-tracking system, and you analyze which parts of the image people are looking at for the most time. The image is of size 1000x1000 (W=H=1000), there are n=1000000 points (multiple people looking at our picture for some time), and k=1000000 queries (k rectangles of various sizes). For each query rectangle, we want to count, how many people looked at it, that is, how many points it contains. However, we state the problem in the abstract form as above, so that a solution to it could be used also for many similar applications.

Solution 1: *A Python program such as: (See the full code online)*

```
[sum(x0[i]<=x[j]<=x1[i] and y0[i]<=y[j]<=y1[i] for j in range(n)) for i in range(k)]
```

Or a C++ program such as: (See the full code online)

```
vector<int> res(k, 0);
for (int i = 0; i < k; i++)
    for (int j = 0; j < n; j++)
        if (x0[i]<=x[j]&&x[j]<=x1[i] && y0[i]<=y[j]&&y[j]<=y1[i])
            res[i]++;
```

We will analyze the time complexity of the C++ program. The last two lines run in time Θ(1) (a constant number of comparisons and additions). The last three lines run in time $n \cdot \Theta(1)$, which is $\Theta(n)$. The last four lines run in $k \cdot \Theta(n)$, which is $\Theta(kn)$. The first line needs to fill a vector with k zeros, which is $\Theta(k)$. The total running time is $\Theta(k)+\Theta(kn)=\Theta(kn)$. Our program uses memory Θ(1) (assuming that we do not count the size of the input and output into our

memory complexity).

Each part of the C++ program has a corresponding part in the Python program (and vice versa)—it is a bit harder to see the steps, and where memory is used, but the complexities are the same.

For the size of data given in our "intended application", we get $k \cdot n$=1000000000000. Thus, our programs would take some time to finish.

***Solution 2:** We could try to count how many times each point appears in the data. In a Python program such as: (See the full code online)*

```
counts = [[0 for _ in range(W)] for _ in range(H)]
for i in range(n):
    counts[y[i]][x[i]] += 1

# Now: counts[b][a] = number of points at the coordinates (a,b)

res = [0 for _ in range(k)]
for j in range(k):
    res[j] = sum(counts[b][a] for a in range(x0[j], x1[j] + 1) for b in range(y0[j], y1[j] + 1))
```

Or in a C++ program such as: (See the full code online)

```
vector<vector<int>> counts(H, vector<int>(W, 0));
for (int i = 0; i < n; i++) counts[y[i]][x[i]]++;

vector<int> res;
for (int i = 0; i < k; i++)
{
    int sum = 0;
    for (int b = y0[i]; b <= y1[i]; b++)
        for (int a = x0[i]; a <= x1[i]; a++)
            sum += counts[b][a];
    res.push_back(sum);
}
```

Regarding the correctness, first observe that, for each coordinate pair (a,b), counts[b][a] is the number of points at this position. Next, observe that, for each i, res[i] equals the variable *sum* that is computed by the two innermost for loops. By the conditions of the for loops, we add up to *sum* all counts[b][a] values for which (a,b) lies within the i-th rectangle. Hence, the two observations imply that res[i] is equal to the number of points contained in the i-th rectangle.

Regarding the running time, note that the first line runs in time $\Theta(W \cdot H)$, the second in time $\Theta(n)$. The remaining part runs in time $\Theta(k \cdot W \cdot H)$ since

- the outermost loop repeats k times,
- the middle for loop repeats $y1[i]-y0[i]=\Theta(H)$ times (to see this, note that, on one hand, $y1[i]-y0[i] \le H=O(H)$, and, on the other hand, in the worst case, $y1[i]-y0[i]=H-1=\Omega(H)$), and
- the innermost for loop repeats $x1[i]-x0[i]=\Theta(W)$ times (by the same reasoning as for the middle loop), each time performing $O(1)$ operations.

Thus, the total time complexity is $\Theta(k \cdot W \cdot H+n)$, and the memory complexity is $\Theta(W \cdot H)$. For our data, it is roughly the same as Solution 1, though the constant (of the upper bound) hidden in the Θ notation might be a bit lower.

Solution 3: However, we could use the counting solution given above to create a much better solution. We first show the idea on a one-dimensional variant of our problem where the array counts is already given:

Simplified Problem:
INPUT: Integer arrays x0[k], x1[k] and counts[W], where all values in x0 and x1 lie between 0 (inclusive) and W (exclusive). For each i, the value counts[i] corresponds to the number of points lying at position i. The arrays x0 and x1 correspond to k intervals on the line; the i-th interval contains every point p with $x0[i] \le p \le x1[i]$.
OUTPUT: An integer array res[k], where, for each j, res[j] is the number of all points lying in the j-th interval. In other words, res[j] = counts[x0[j]] + counts[x0[j]+1] + ... + counts[x1[j]] = $\sum_{x0[j] \le i \le x1[j]} counts[i]$.

This task can be done in time $\Theta(k \cdot W)$ as above. However, we can do it faster:

Compute the vector cumsum of size **W+1**, where cumsum[i] is the sum of counts[j] for j<i, that is, cumsum[i]=counts[0]+counts[1] + ... + counts[**i-1**]. In other words, cumsum[i] is the number of points that are lying to the left of position i! This vector can be easily computed in time $O(W)$ by setting cumsum[0]=0, cumsum[1]=counts[0] and, for each i > 1, cumsum[i]=cumsum[i-1]+counts[i-1]. Once we have computed cumsum, res[i] can be computed for each i in $O(1)$ time using the formula res[i] = cumsum[x1[i]+1] - cumsum[x0[i]]. In total, we get thus a running time of $O(k+W)$.

Note that, in the formula above, we have to write cumsum[x1[i]**+1**] and not just cumsum[x1[i]]. The reason is that, by our definition, cumsum[x1[i]] does not count the points on the right-most position x1[i] of the i-th interval. Also note that it suffices to compute cumsum until cumsum[W] since, for all i, we have x1[i]<W.

Here the corresponding Python code: (See full code online)

```
counts = [0 for _ in range(W)]
for i in range(n):
    counts[x[i]] += 1

cumsum = [0 for _ in range(W + 1)]
for p in range(1, W + 1):
    cumsum[p] = cumsum[p - 1] + counts[p - 1]

res = [0 for _ in range(k)]
for j in range(k):
    res[j] = cumsum[x1[j] + 1] - cumsum[x0[j]]
```

Our main problem is solved in the same way, but in two dimensions. On the slides you find an animation describing the idea. In the following, we define—slightly different than in the lecture—cumsum[*b*][*a*] to be the number of all points that are contained in the *black rectangle area* defined as follows: one corner is (*a-1*, *b-1*), the opposite corner is the origin. Thus, a point belongs to this black area of cumsum[b][a] exactly then when its x-coordinate is smaller than *a* and its y-coordinate is smaller than *b*.
To simplify the code, we define counts[b][a] to be the number of points at position (b-1,a-1). Also, instead of creating a new array for cumsum, we reuse the array counts.

First, the corresponding code in C++: (See the full code online as well as an optimization here; you find there more explanations.)

```
vector<vector<int>> counts(H + 1, vector<int>(W + 1, 0));
for (int i = 0; i < n; i++)
    counts[y[i] + 1][x[i] + 1]++;

// Compute cumulative sums in each row
for (int b = 0; b <= H; b++)
{
    for (int a = 1; a <= W; a++)
    {
        counts[b][a] += counts[b][a - 1];
    }
}

// Compute cumulative sums in each column
for (int a = 0; a <= W; a++)
{
    for (int b = 1; b <= H; b++)
    {
        counts[b][a] += counts[b - 1][a];
    }
}

vector<int> res;
for (int j = 0; j < k; j++)
    res.push_back(
        counts[y1[j] + 1][x1[j] + 1] + counts[y0[j]][x0[j]] - counts[y1[j] + 1][x0[j]] - counts[y0[j]][x1[j] + 1]);
```

This code runs in time $\Theta(k+W \cdot H+n)$, thus, for our "intended application", much faster than the previous algorithms. The memory complexity is still $\Theta(W \cdot H)$.

This technique is called *cumulative sums*.

In Python, cumulative sums can be computed with the function cumsum in the library numpy: (See the full code online with a visualization; here you find the corresponding Python code without numpy.)

```
counts = np.zeros([H+1, W+1], dtype=int)
for i in range(n):
    counts[y[i]+1][x[i]+1] += 1
np.cumsum(counts, axis=0, out=counts)
np.cumsum(counts, axis=1, out=counts)
return [counts[y1[j]+1][x1[j]+1] + counts[y0[j]][x0[j]] - counts[y0[j]][x1[j]+1] - counts[y1[j]+1][x0[j]] for j in range(k)]
```

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:18

◄ Lecture 4 - recording

Przejdź do...

Lecture 5 ►